



Display Control

Complete Technical, Security, Workflow, Operations, and Management Handbook

Raspberry Pi digital signage

Multi-tenant platform

React + ASP.NET Core

PostgreSQL RLS

mTLS + signed offline licences

Prepared: 21 August 2026

Repository: Display Control working title, version 0.1 development line

Audience: Project manager, supervisor, developer, security reviewer, and operations team

Purpose: Explain what the application does, how it works, the technologies used, important security decisions, known weaknesses, deployment requirements, and what remains before production.

Document scope and evidence basis

Important: This handbook describes the application-owned source code, configuration, tests, deployment assets, and project documentation present in the repository on 21 August 2026. It does not treat planned work as completed work. Third-party dependency source, Git internals, compiled output, local secrets, runtime data, and delivery artifacts are outside this review.

The repository contains roughly 300 application-owned files and about 52,000 lines when generated Entity Framework migrations and dependency lockfiles are included. The implementation is a serious secure vertical slice, but its own project status correctly says that it is not yet production-ready.

The repository records the following most recent local verification: 85 .NET tests, 7 React tests, and 5 Playwright/axe browser scenarios passing. These figures are historical evidence recorded by the project; they were not rerun while preparing this handbook.

Table of contents

1. Executive summary	12. Content, playlists, and scheduling
2. Business problem and value	13. Synchronization, cache, and playback
3. Actors, roles, and permissions	14. Email notification workflow
4. System architecture	15. Security model and controls
5. Technology stack	16. Known vulnerabilities, limitations, and risks
6. Repository and code organization	17. Testing and evidence
7. Data model	18. Development and deployment
8. Human identity workflows	19. Operations and troubleshooting
9. Tenant administration workflows	20. Current status and production gap
10. Raspberry Pi identity and heartbeat workflows	21. Manager talking points and questions
11. Licensing and offline authorization	22. Glossary

1. Executive summary

Display Control is a secure multi-customer digital-signage platform for Raspberry Pi screens. A central administration application lets authorized users manage customer organizations, users, screens, licences, media, playlists, groups, and schedules. Each Raspberry Pi independently connects to the central service, proves its identity with a client certificate, downloads only the content it is authorized to use, verifies every file, and displays it through a local Chromium kiosk.

The system is built around a fail-closed rule: if tenant identity, device identity, licence status, signed authorization, trusted time, media approval, or file integrity cannot be proven, the device must not display protected content. It shows **Not licensed** or another safe operational state instead.

The one-sentence architecture

The server decides and signs; the Pi agent verifies and enforces; the local React player only renders what the agent has authorized.

What is already implemented

- Platform and tenant administration with invitation-only accounts and role-based access.
- TOTP multifactor authentication, recovery codes, revocable server-side sessions, CSRF protection, and rate limiting.
- Strict multi-tenant database isolation using application checks, Entity Framework filters, same-tenant foreign keys, and forced PostgreSQL Row-Level Security.
- One-use Raspberry Pi enrollment, locally generated ECDSA keys, client certificates, mTLS authentication, certificate rotation, and revocation.
- Device inventory and heartbeat reporting including serial, host, OS, software versions, disk, IP, and MAC information.
- Per-device time-limited licences with renewal, suspension, revocation, transfer, and signed bounded offline leases.
- Private content upload, file-signature inspection, SHA-256 hashing, ClamAV scanning, quarantine, approval, and immutable content versions.
- Immutable playlists, device groups, direct/group assignments, priorities, and bounded scheduling.
- Verified/resumable content downloads, local cache activation, and image/video/text playback.
- Raspberry Pi systemd services, restricted service accounts, field-test packaging, and a virtual-device workflow.

What is not yet proven

- Production infrastructure and operational readiness.
- Physical Pi 4/5 playback, reboot, power-loss, clock, disk-pressure, and endurance evidence.
- Real SMTP delivery and complete live-backend browser journeys.
- Production monitoring, backup/restore, load testing, penetration testing, and signed agent updates.

2. Business problem and value

The platform solves a common remote-screen problem: an organization wants to control content shown on Raspberry Pi displays located on different networks without opening inbound access to those customer networks.

Business capabilities

- Create isolated customer spaces.
- Delegate access to customer administrators and content managers.
- See which screens exist, where they are logically assigned, their network inventory, last contact, player health, and licence state.
- Approve only safe content and publish stable playlists.
- Target content to one device or a group, immediately or during a bounded time window.
- Stop playback by suspending a device, tenant, licence, or certificate.
- Allow limited offline playback without granting indefinite authorization.
- Retain security/business audit evidence.

Design value

The architecture avoids LAN discovery and inbound router rules. Every Pi initiates outbound HTTPS to the public service. This is easier to deploy through NAT and usually safer than allowing the platform to reach into customer networks.

The system also separates content possession from content authorization. A file may remain cached on disk, but the player cannot access it after authorization is removed because the agent clears the local manifest and asset mapping.

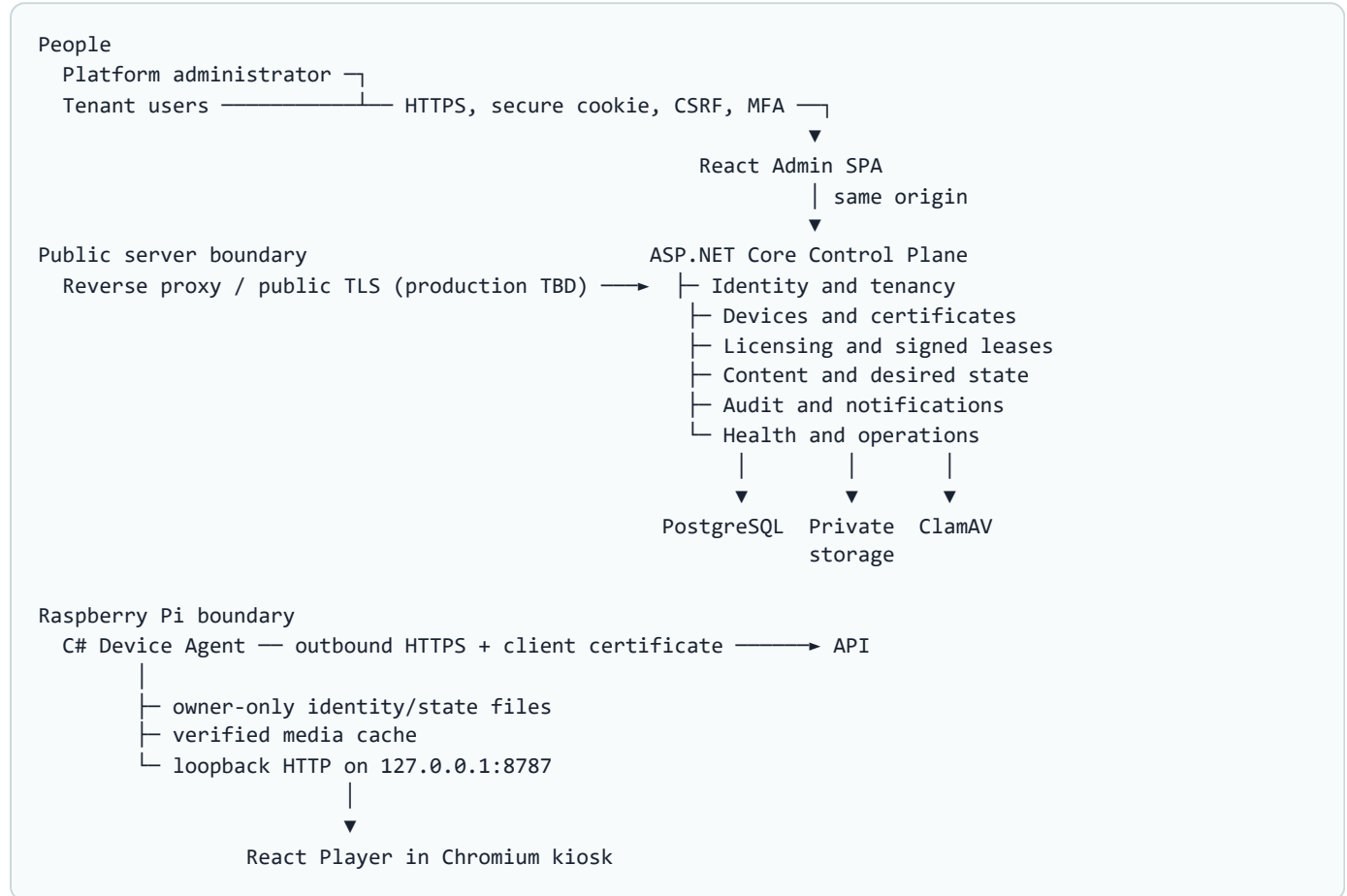
3. Actors, roles, and permissions

Actor	Responsibility	Important restrictions
Platform administrator	Creates and manages customer tenants, including suspension and archiving.	Uses MFA; sensitive tenant operations require recent MFA. Does not automatically become a tenant user.
Tenant administrator	Manages members, devices, groups, certificates, enrolment codes, and licences for one tenant.	Bound to one tenant. Administrative access requires MFA; selected actions require a fresh MFA proof.
Content manager	Uploads, approves, archives, and publishes content/playlists/assignments.	Cannot administer tenant membership or licences.
Viewer	Reads tenant dashboards and operational state.	No mutation permission.
Raspberry Pi agent	Enrolls, authenticates, reports inventory, receives authorization, synchronizes content, and controls playback.	Uses a device certificate, never a human cookie. It receives only device-scoped content.
Local React player	Renders a sanitized local state and local content URLs.	Has no device private key, backend token, storage credential, or unrestricted filesystem path.
Notification worker	Sends invitation and password-recovery messages.	Uses a separate restricted database identity and only processes protected notification rows.

No public signup: users enter through a one-use invitation. The very first platform administrator is created through a separate one-time bootstrap path protected by a configured token digest.

4. System architecture

The selected architecture is a **modular monolith**. The backend runs as one ASP.NET Core application and one database, but responsibilities are separated into domain, application, infrastructure, HTTP, and background-worker components.



Why a modular monolith?

- One backend is easier to deploy, debug, migrate, and explain than many services.
- Transactions can atomically update business state and audit/notification intent.
- The project still keeps explicit boundaries so a future split remains possible.
- It fits the scope of an internship/project while preserving serious security controls.

Important architectural nuance

The intended layers exist, but the current Application project is relatively thin. It mainly contains interfaces, token/manifest codecs, file inspection, and storage/security contracts. A meaningful amount of use-case orchestration is currently implemented directly in API controllers and focused API services. This is acceptable for the current modular monolith but should be considered if business workflows become larger or need reuse outside HTTP.

Network model

- Browsers communicate with the admin application and same-origin API.
- Pis communicate outward to device API routes over HTTPS with a client certificate.

- No public service connects inbound to a Pi.
- The player server listens only on 127.0.0.1:8787.
- Production documentation recommends separate admin and device hostnames so client-certificate handling does not complicate human traffic.

5. Technology stack

Area	Technology/version	Purpose	Operational meaning
Backend	.NET SDK 10.0.203; ASP.NET Core 10.0.7; C# 14	HTTP APIs, security middleware, health endpoints, and workers.	Exact SDK and package lines are pinned.
Domain/data	Entity Framework Core 10.0.7; Npgsql 10.0.3	Domain persistence, schema mapping, and migrations.	Migrations are generated but security SQL must be reviewed manually.
Database	PostgreSQL 18.4	Identity, tenant data, devices, licensing, schedules, audit, and queues.	Forced RLS is a core security boundary, not merely a query convention.
Web applications	React 19.2.8; TypeScript 6.0.2; Vite 8.2.1	Administration and kiosk UIs.	No heavy state-management or router dependency; components call APIs directly.
Pi runtime	Self-contained .NET Linux ARM64; Chromium; systemd	Background synchronization and local kiosk.	Target is Raspberry Pi OS 64-bit on Pi 4/5.
Malware scanning	ClamAV 1.4.5	Scans quarantined uploads using the clamd streaming protocol.	Scanner unavailability keeps content restricted.
Cryptography	ECDSA P-256, ES256, SHA-256, HMAC-SHA256, ASP.NET Data Protection, RFC 6238 TOTP	Device identity, signed leases, token digests, sensitive payload protection, and MFA.	Private key files and Data Protection keys must be handled as production secrets.
Containers	Docker Compose	Local PostgreSQL and ClamAV.	Compose is a development baseline, not the final production topology.
Tests	xUnit, Testcontainers, Vitest, Testing Library, Playwright 1.61.1, axe-core	Domain, database, protocol, component, browser, and accessibility checks.	Mocked browser tests do not replace a live staging journey.
CI	GitHub Actions on Ubuntu 24.04	Restore, build, test, audit, lint, type-check, browser checks, and documentation verification.	Workflow exists; remote evidence remains listed as open in project status.

Dependency policy

NuGet versions are centrally pinned, npm uses an exact lockfile, the .NET SDK is locked by global.json, container tags are exact, warnings are treated as errors, nullable analysis is enabled, and builds are deterministic. This reduces “works on my machine” variation and supply-chain drift.

6. Repository and code organization

Path	What it contains	What to understand
apps/admin-web	Authentication views, platform dashboard, tenant operations dashboard, styling, and component tests.	French-first UI. It exposes the core demonstration workflow but not every API operation.
apps/player-web	Kiosk player, safe states, playlist rotation, media elements, and tests.	Polls only the local agent; defaults to Not licensed.
src/DisplayControl.Api	Startup, dependency injection, security policies/middleware, controllers, desired-state services, and notification worker.	Main control plane and workflow orchestration.
src/DisplayControl.Domain	Tenant, identity, device, licence, content, playlist, schedule, and audit state/rules.	Pure C# rules independent of HTTP and EF implementation details.
src/DisplayControl.Application	Ports/interfaces, content inspection, manifest codec, lease codec, and claim contracts.	Defines boundaries used by API, infrastructure, and agent.
src/DisplayControl.Infrastructure	EF DbContext/migrations, Identity store, PostgreSQL, cryptographic services, storage adapter, ClamAV adapter.	Implements external/persistent concerns.
src/DisplayControl.DeviceAgent	Enrollment, certificate rotation, heartbeat, trusted clock, cache, local player server, and inventory.	Security authority on the Pi.
tests	Domain, agent, integration, PostgreSQL RLS, crypto, storage, and browser tests.	Evidence is layered; each test type proves a different boundary.
deploy	Pi installer/systemd units and PostgreSQL runtime/notification role templates.	Security-sensitive deployment material.
scripts	Development keys, platform bootstrap, virtual device, field environment, and Pi bundle scripts.	Convenience for local/field testing; not a production secret manager.
docs	Requirements, ADRs, architecture, threats, testing, operations, status, and report plan.	The repository is documentation-heavy and designed for traceability.

7. Data model

Most identifiers are application-generated UUIDs. Authorization timestamps are UTC. Licence and schedule windows use an inclusive start and exclusive end: [start, end). Tenant-owned rows contain a non-null tenant_id even when the tenant could be inferred through another row.

Identity and tenancy

- **Tenant:** customer organization, slug, timezone, state, and concurrency token.
- **ApplicationUser:** global ASP.NET Identity account with verified email and home-tenant locator.
- **TenantMembership:** user-to-tenant role and lifecycle.
- **Invitation:** target email, intended role, expiry, creator, and token digest.
- **UserSession:** session digest, expiry, tenant binding, MFA state, revocation, and safe client fingerprints.
- **UserMfaSecret/UserRecoveryCode:** protected TOTP secret and hashed one-use recovery codes.
- **IdentityNotification:** protected invitation/reset payload and retry state.

Devices

- **Device:** tenant-owned Pi, lifecycle, reported serial, host inventory, version, disk, last seen, and player state.
- **EnrollmentToken:** short-lived, one-use capability with optional expected serial/device binding.
- **DeviceCertificate:** certificate DER, fingerprints, public-key digest, validity, state, and rotation lineage. No private key is stored by the server.
- **DeviceHeartbeat:** sequence, inventory snapshot, applied desired state, player/disk state, public source address, and correlation ID.
- **DeviceNetworkInterface:** latest interface, normalized MAC, local addresses, and observation time.

Licensing

- **DeviceLicense:** device, validity window, control state, latest issued lease boundary, transfer destination, and concurrency token.
- **LicenseEvent:** append-oriented lifecycle event with actor, reason, before/after snapshots, and device lineage.

Content and scheduling

- **ContentAsset:** logical title/media kind and lifecycle.
- **ContentVersion:** immutable storage reference, byte length, SHA-256, MIME, metadata, scan result, and approval.
- **Playlist/PlaylistVersion/PlaylistItem:** logical playlist plus immutable ordered published version.
- **DeviceGroup/DeviceGroupMember:** tenant group membership.
- **DeviceAssignment/GroupAssignment:** target, playlist version, priority, UTC window, and presentation timezone.
- **DesiredState/DesiredStateAsset:** immutable, device-specific compiled manifest with monotonic version and manifest digest.

Operational records

- **TenantAuditEvent:** actor, action, target, outcome, reason, correlation ID, and bounded JSON details.
- **OutboxMessage:** generic transactional-outbox shape.
- **SystemKeyMetadata:** public key metadata and activation/retirement dates.

8. Human identity workflows

8.1 First platform administrator

- 1 Deployment generates a high-entropy bootstrap token and stores only its SHA-256 digest in API configuration.
- 2 An operator calls the platform-bootstrap endpoint with email, display name, password, and token.
- 3 The endpoint is rate-limited to three attempts per hour per source address.
- 4 If a platform administrator already exists, bootstrap becomes logically unavailable.
- 5 The created account must enroll TOTP at first sign-in.

8.2 Session bootstrap and CSRF

- 1 The browser loads the administration SPA and calls GET /api/v1/session.
- 2 The API returns anonymous/authenticated session state and issues a protected CSRF cookie/token pair.
- 3 The React app keeps the returned CSRF token in memory and sends it in X-CSRF-TOKEN for unsafe requests.
- 4 The global antiforgery filter rejects unsafe cookie-based requests without a valid token before business work occurs.

8.3 Sign-in and MFA

- 1 Email/password are processed through ASP.NET Core Identity with neutral failure behavior and rate limiting.
- 2 A server-side UserSession is created; the browser receives an opaque secure cookie.
- 3 If an administrator has no confirmed MFA secret, the session enters mfa_enrollment.
- 4 The user registers a generated TOTP secret in an authenticator and confirms a six-digit code.
- 5 Ten one-use recovery codes are displayed once. Only digests are stored.
- 6 Future sign-ins enter mfa_pending until TOTP succeeds.
- 7 Each accepted TOTP time step is persisted so the same code window cannot be replayed.

8.4 Session validation

Authentication is not trusted only because a cookie exists. On each request, middleware verifies the session digest, user, security stamp, expiry, revocation state, home tenant, tenant state, membership state, and current role. A changed role or suspended membership is therefore enforced without waiting for the eight-hour cookie expiry.

8.5 Recent MFA

Sensitive actions require an MFA proof no older than ten minutes. The user can enter a current code through the step-up endpoint. Examples include role changes, certificate revocation, licence transfer, recovery-code regeneration, global sign-out, and sensitive platform tenant actions.

8.6 Password recovery

- 1 The forgot-password response is identical whether or not an account exists.
- 2 For an eligible account, a short-lived reset token is wrapped in a protected notification payload and queued.
- 3 The notification worker builds an encoded public HTTPS reset link and delivers it via TLS-enabled SMTP.
- 4 Successful password reset updates the security stamp and revokes prior sessions.

9. Tenant administration workflows

9.1 Creating a customer

- 1 A platform administrator with recent MFA submits tenant name, slug, timezone, first-admin email, and first-admin role.
- 2 The API creates the tenant and a tenant-bound one-use invitation.
- 3 The raw invitation token is displayed or sent only once; the database stores a digest.
- 4 The invited administrator accepts the link, chooses a display name/password, then enrolls MFA.

9.2 Tenant lifecycle

- **Active:** normal user and device operation.
- **Suspended:** human sessions are revoked and subsequent device authentication/heartbeats are rejected.
- **Archived:** terminal state; the tenant cannot be modified or reactivated.

9.3 Member lifecycle

A tenant administrator can invite members, list them, change roles, suspend/reactivate/remove access, and see MFA state. Role changes and membership restrictions revoke affected sessions. Domain rules preserve lifecycle consistency and project requirements include protecting the last-administrator invariant.

10. Raspberry Pi identity and heartbeat workflows

10.1 Enrollment code creation

A tenant administrator creates a pending device and a high-entropy, one-use enrollment code. The code can be bound to an expected serial number and normally expires after 15 minutes. Only its HMAC-SHA256 digest is persisted.

10.2 Device enrollment

- 1 The Pi reads its serial from `/proc/cpuinfo`, gathers host/network/disk inventory, and generates an ECDSA P-256 private key locally.
- 2 The private key is written to an owner-only file. The Pi sends only a certificate-signing request, enrollment code, and inventory.
- 3 The API rate-limits the request, identifies the tenant from the capability token, checks expiry/use/attempt count/expected serial, and atomically consumes it.
- 4 The certificate authority issues a client certificate containing tenant and device identity extensions.
- 5 The response includes the public CA, certificate, server UTC, and pinned licence-verification public key.
- 6 The Pi verifies the certificate matches its private key, validates the chain against the returned CA, validates the signing public key, and persists state atomically.
- 7 Only after successful persistence does it delete the temporary enrollment-code file.

10.3 Device authentication

Device API routes require a presented client certificate. Custom middleware rebuilds the chain against the configured device CA, reads the identity extensions, verifies certificate time and cryptographic properties, then verifies the certificate record, device state, and tenant state in PostgreSQL. It creates a device-only principal containing tenant, device, certificate, and mTLS claims.

10.4 Certificate rotation

When the current certificate has 30 days or less remaining, the agent creates a separate next private key and CSR. The API issues a replacement certificate and limits overlap of the old certificate. The agent verifies that the CA has not changed and that the certificate matches the pending key. State is written before the key is promoted, and recovery logic handles interruption between those operations.

10.5 Heartbeats

- 1 Every configured interval (normally 30 seconds plus 0–2 seconds jitter), the agent collects fresh inventory.

- 2 It sends an increasing sequence number, player state, applied desired-state version, disk information, and interface snapshot.
- 3 The server rejects repeated/out-of-order sequence numbers.
- 4 The current device row is updated and a heartbeat history row is appended.
- 5 Network-interface rows are replaced with the current normalized snapshot.
- 6 The server evaluates licence and desired state, then returns either denial or a signed lease.

11. Licensing and offline authorization

11.1 Licence states

Control/effective state	Meaning	Playback result
Future	Current server time is before the inclusive licence start.	Not licensed.
Active / enabled	Inside [validFromUtc, expiresAtUtc) and not administratively restricted.	Eligible for signed lease.
Expired	Current time is at or after the exclusive end.	Not licensed.
Suspended	Temporary administrative denial.	Not licensed after next authoritative contact; existing offline lease remains bounded.
Revoked	Terminal administrative denial.	No new authorization.
Transfer pending	Moving entitlement while the source may still have an outstanding lease.	Destination waits until the source lease boundary.

11.2 Signed offline lease

The server signs leases with ECDSA P-256 using ES256. A lease binds at least:

- tenant, device, certificate, and licence identifiers;
- issue, not-before, and not-after timestamps;
- desired-state identifier/version and manifest SHA-256, when content exists;
- a signing-key identifier, algorithm, schema version, and unique token identifier.

The expiry is the earliest applicable boundary:

```
lease expiry = minimum of:
  authenticated server time + configured offline allowance (maximum 24 hours)
  actual licence expiry
  device certificate expiry
  assignment/schedule end, when one exists
```

11.3 Trusted time

The agent anchors time to the most recent authenticated server UTC plus monotonic elapsed time during the process lifetime. It rejects an authenticated server time that rolls back beyond tolerance. If the network fails during the same boot, a valid cached lease can continue to authorize already-active content until expiry.

Deliberate availability trade-off: after an agent restart with no authenticated server contact, the in-memory monotonic anchor is unavailable. The agent fails closed rather than trusting the Pi wall clock. Cached files may exist, but playback is not authorized until server time is re-established.

11.4 Licence transfer

A transfer cannot instantly authorize both source and destination. The destination licence begins no earlier than the latest issued source lease expiry. This prevents a normal transfer operation from creating overlapping authorized playback.

12. Content, playlists, and scheduling

12.1 Upload and inspection

- 1 The content manager selects title, declared media kind, and file.
- 2 The API enforces configured request/object size limits. The current default object ceiling is 256 MiB.
- 3 The file inspector streams the bytes, calculates SHA-256, and validates actual signature against the declared kind.
- 4 PNG/JPEG dimensions are bounded; text must be valid UTF-8 without unsafe control characters; WebP and MP4 signatures are recognized.
- 5 The private object store writes to a temporary file and atomically publishes it without overwrite.
- 6 ClamAV scans the stored bytes through a bounded streaming protocol.
- 7 Clean content becomes a draft; infected content is rejected; scanner failure keeps it quarantined.
- 8 A content manager must explicitly approve a clean draft before playlist use.

12.2 Private storage

Media bytes are not stored in PostgreSQL and are not placed in a public web root. Code addresses objects using a key made only from normalized tenant and object UUIDs. Browser filenames and device inputs cannot become filesystem paths. The local adapter rejects links/reparse points, length mismatches, oversize streams, and overwrites.

12.3 Playlists

A logical playlist is created with ordered items that refer to approved content versions. Publication freezes a playlist version. The immutable snapshot records content identifiers, order, image/text durations, video looping, and presentation data.

12.4 Assignments and schedule resolution

- A playlist version can be assigned directly to a device or to a device group.
- Bounds are optional UTC instants using `[start, end)`.
- Priority is between -1000 and 1000.
- Direct-device assignment always outranks group assignment.
- Within the same class, highest numeric priority wins.
- If the two leading active candidates have equal priority, the resolver throws an ambiguity error rather than choosing arbitrarily.
- Recurring schedules are intentionally outside v1.

12.5 Desired-state compilation

Publishing an assignment verifies that its playlist is immutable/published and that every referenced content version is still clean and approved. The server flattens it into a device-specific desired state with a monotonic version and canonical manifest SHA-256. That digest is later included in the signed lease.

13. Synchronization, cache, and playback

13.1 Authorization response

A licensed heartbeat returns server time, retry advice, a signed lease, the licence expiry, and either `LicensedNoContent` or an available desired-state identifier/version. An unlicensed heartbeat returns no lease and a not-licensed desired-state summary.

13.2 Agent validation

- 1 Verify authenticated server time does not move backward unexpectedly.
- 2 Verify lease key identifier, algorithm, and public key match the pinned verification key.
- 3 Verify the ES256 signature and tenant/device/certificate/time bindings.
- 4 Verify the desired-state response matches the lease payload.
- 5 Persist authorization state atomically before synchronization.

13.3 Manifest and asset synchronization

- 1 The player enters Synchronizing and its previous manifest is removed from the browser-facing state.
- 2 The agent requests the authorized manifest through the device API.
- 3 Manifest size, ID, version, time bounds, asset count/order, unique IDs, media kinds, durations, lengths, and SHA-256 fields are validated.
- 4 The canonical manifest SHA-256 must match the value signed into the lease.
- 5 Existing cached objects are rehashed before reuse.
- 6 Missing assets download into a private staging directory.
- 7 Interrupted downloads resume using HTTP Range. Content-Range and response length are verified.
- 8 The agent hashes the entire final object and rejects length/digest mismatch.
- 9 Verified objects and manifest are atomically promoted; stale inactive manifests are removed.
- 10 The agent exposes a sanitized in-memory manifest to the local player and records the applied version.

13.4 Cache pressure

The default cache ceiling is 4 GiB with a 256 MiB free-disk reserve. Before download, the agent calculates required object and filesystem space. It removes least-recently-written inactive objects while protecting active hashes. If the replacement cannot fit safely, synchronization fails without partially authorizing it.

13.5 Local player

The agent hosts static player files and four loopback endpoints: health, player state, manifest, and authorized asset bytes. Assets are served only when present in the active in-memory mapping; physical paths are never returned to React.

Player state	Meaning	UI
notLicensed	No valid authorization or an error makes authorization uncertain.	Exact message “Not licensed”.
noContent	Licence is valid but no assignment resolves.	“No content assigned”.
synchronizing	Authorization exists but active content is not ready.	Synchronization indicator.
ready	Manifest and assets are verified and active.	Ordered image, video, or text playback.

The React player polls local state every five seconds. Images/text advance by configured duration. Non-looping video advances on its ended event; looping video remains on the same item. If agent state or manifest loading fails, React returns to the safe default.

14. Email notification workflow

Notification delivery is optional and starts only when enabled by configuration. Invitation and password-reset payloads are protected using an independent ASP.NET Core Data Protection purpose before database insertion.

- 1

The web transaction enqueues a protected notification but its restricted runtime role cannot read notification payloads back.
- 2

A separate worker connection uses a separately restricted PostgreSQL role.
- 3

The worker sets an explicit notification-delivery context and claims one due row using `FOR UPDATE SKIP LOCKED`.
- 4

It verifies the protection scheme and unprotects only the selected payload.
- 5

It creates the appropriate invitation/reset message and sends it through TLS-enabled SMTP.
- 6

Success marks the row processed. Failure stores only a safe error code and schedules progressive retry.

Status: the worker and security behavior exist and are tested at code level, but real delivery through a staging SMTP provider is still an open acceptance item.

15. Security model and controls

15.1 Defense in depth for tenant isolation

1. The browser does not choose an authoritative tenant; the authenticated session does.
2. Authorization compares a tenant route to current membership.
3. Middleware creates a scoped trusted tenant context.
4. Tenant-owned database work runs inside a transaction that uses PostgreSQL `set_config('app.tenant_id', ..., true)`.
5. The final `true` makes context transaction-local so pooled connections do not leak tenant state.
6. EF query filters default-deny when context is missing.
7. Forced PostgreSQL RLS policies enforce the same boundary in the database.
8. Composite foreign keys prevent cross-tenant relationships.
9. The runtime database role is neither owner, superuser, nor `BYPASSRLS`.

15.2 Human security controls

- Invitation-only onboarding; one-use expiring tokens stored as digests.
- Unique verified email, password hashing configured to 210,000 iterations, and account lockout.
- TOTP MFA, time-step replay guard, one-use recovery codes, and recent-MFA step-up.
- Opaque server-side sessions with security-stamp validation, idle/absolute expiry, and revocation.
- Secure `__Host-` cookies: HTTPS-only, `HttpOnly`, `SameSite Strict`, root path.
- CSRF protection on unsafe cookie-based routes.
- Neutral authentication/recovery errors and endpoint-specific rate limits.

15.3 Device security controls

- Human and device identity systems are separate.
- Enrollment token is only a bootstrap capability, not a long-term credential.
- Device private key originates and stays on the Pi.
- Only ECDSA P-256 CSRs/certificates are accepted.
- mTLS identity is bound to tenant, device, certificate record, device state, and tenant state.
- Certificate rotation uses proof of possession and bounded overlap.
- Heartbeat sequence prevents basic replay/out-of-order processing.
- Signed leases bind authorization to certificate and manifest.

15.4 Content security controls

- Private storage with opaque UUID-derived keys and no public directory.
- Bounded streaming rather than trusting complete in-memory uploads.
- Declared type must match magic bytes; invalid UTF-8/control text is rejected.
- Malware scan is fail-closed; approval is a separate explicit action.
- Published content/playlist/manifests are immutable.
- Every downloaded object and manifest is checked with SHA-256.
- HTML, JavaScript, executable uploads, and scriptable SVG are not supported.

15.5 Web and service hardening

- Security headers: no server banner, HSTS outside development/testing, CSP, nosniff, no-referrer, and restrictive permissions policy.
- No explicit permissive CORS configuration; the admin design is same-origin.
- OpenAPI is exposed only in development.
- API errors use bounded Problem Details rather than raw exception content.
- Pi services run under restricted accounts with systemd hardening and no extra capabilities.
- Chromium keeps its sandbox; the launcher does not use `--no-sandbox` or a remote-debugging port.
- State/cache code rejects symbolic links and Windows reparse points.

16. Known vulnerabilities, limitations, and risks

“Vulnerability” should be used carefully. Some items below are exploitable security risks, some are missing production controls, and others are deliberate availability or scope limitations. They should all be understood before a production claim.

ID / severity	Risk or limitation	Why it matters	Current mitigation	Recommended action
R-01 High	Physical/root compromise of a Pi	A person with sustained root or physical control can modify the agent/player or copy software-held keys and cached content.	Managed-appliance assumption, separate accounts, owner-only files, systemd hardening.	Explicitly accept the threat model or add hardware-backed keys, secure/measured boot, locked OS/admin access, and physical controls.
R-02 High before production	No production environment evidence	Local behavior does not prove DNS/TLS edge, secret handling, restricted roles, scanning, backups, recovery, or operational isolation.	Detailed production requirements and deployment role templates exist.	Build a production-like staging environment and execute release gates.
R-03 High before production	No physical Pi acceptance	ARM64, HDMI, Wayland/X11, GPU decode, reboot, power loss, clock behavior, disk pressure, and long-running kiosk stability remain unproven.	Linux ARM64 bundle, installer, services, and field runbook exist.	Run Pi 4 and Pi 5 field/soak matrix and retain evidence.
R-04 Medium	Single-writer local object store	The filesystem adapter is unsafe for multiple API replicas and depends on deployment volume encryption/backup.	Immutable/no-overwrite adapter and explicit single-writer documentation.	Select and test a private S3-compatible production adapter for multi-node/cloud deployments.
R-05 Medium	Browser E2E suite intercepts API responses	It proves UI behavior, request shape, CSRF use, and accessibility for covered views, not the full deployed system.	Real PostgreSQL/API integration tests exist separately.	Add end-to-end browser journeys against staging API, PostgreSQL, storage, scanner, email, and mTLS device flows.
R-06 Medium	Media validation is not complete codec validation	MP4 recognition checks the container signature but does not yet prove H.264/AAC profile/level or Pi hardware compatibility. Complex/polyglot files need adversarial testing.	Allowlist, size/signature checks, SHA-256, private storage, and ClamAV.	Add isolated ffmpeg/codecs validation, malicious/polyglot corpus tests, and real Pi playback evidence.
R-07 Medium	SMTP is not externally validated	Invitation/reset usability, TLS/authentication, provider policy, bounce handling, and deliverability are unknown.	Protected payload, separate worker role, TLS SMTP, retry/backoff.	Use a staging provider and add dead-letter, retention, metrics, alerts, bounce/support process.

ID / severity	Risk or limitation	Why it matters	Current mitigation	Recommended action
R-08 Medium	No production observability or DR proof	Failures may go unnoticed and data/content may not be recoverable consistently.	Generic health endpoints and safe structured logs exist.	Add metrics/log aggregation/alerts, coherent DB+object backups, encrypted retention, and restoration drills.
R-09 Medium	No load/capacity evidence	Expected tenant/device fleet size is still an open decision; 30-second heartbeats and synchronization bursts may overload components.	Configurable intervals, jitter, bounded requests, and query indexes.	Freeze capacity assumptions and test above expected peak with k6 or equivalent.
R-10 Medium	Multi-instance publication race	Desired-state version generation uses current maximum + 1; concurrent publication across server instances requires stronger serialization.	Database uniqueness and local workflow checks; project status already flags publication locking.	Add transaction/advisory locking or a database sequence and prove concurrency behavior.
R-11 Medium	Human-command idempotency is incomplete	Retries after timeouts can repeat critical mutations.	Some device operations are replay-safe and concurrency tokens protect stale writes.	Generalize Idempotency-Key to critical human commands and persist results.
R-12 Medium	Signed agent update/rollback incomplete	Operational updates could become a software supply-chain or availability risk.	Release archive SHA-256 is verified during field installation; releases use atomic current symlink.	Add signed metadata, trusted key rotation, rollback protection, staged rollout, and recovery procedure.
R-13 Low/Medium	Offline playback fails after cold restart without server time	A legitimate device may show Not licensed during an outage after reboot even if a lease file has not expired.	Deliberate fail-closed design avoids trusting an uncertain wall clock.	Keep as accepted safety behavior or design a hardware-backed trusted-time solution.
R-14 Low/Medium	Admin UI does not expose every backend action	Some certificate/device/licence lifecycle operations require direct API use or additional screens.	Core demonstration path is available; API contract is richer.	Complete UX coverage and role/accessibility testing before user acceptance.
R-15 Low/Medium	Media size decision inconsistency	Runtime default is 256 MiB while open project decisions mention a provisional 2 GiB video ceiling.	Runtime validates configured ceiling between 1 KiB and 2 GiB.	Freeze per-media limits, align documentation/config/UI, and load-test streaming behavior.

ID / severity	Risk or limitation	Why it matters	Current mitigation	Recommended action
R-16 Medium	Security tooling/review not complete	A green unit/integration suite does not replace secret scanning, SBOM, container scan, DAST, ASVS review, or penetration test.	Pinned dependencies, npm audit, analyzers, threat model, and security tests.	Add Gitleaks/Trivy/SBOM/ZAP or equivalents and an independent penetration test.
R-17 Configuration critical	TLS client certificate handling depends on correct edge deployment	Kestrel transports optional client certificates and custom middleware is the authoritative validator. A proxy must preserve the intended certificate trust path without accepting spoofed headers.	Middleware validates chain, extension, DB record, device, certificate, and tenant. Real Kestrel mTLS tests exist.	Document/test the final reverse proxy end to end; reject untrusted forwarding headers and verify absent/invalid/revoked certificate behavior.
R-18 Low/Medium	Audit coverage and retention require final review	Audit entities and many controller events exist, but every sensitive success/failure and operational retention/export requirement must be proven.	Append-oriented audit model with safe bounded details and correlation IDs.	Build an endpoint/action audit matrix and retention/export/integrity process.

Do not tell stakeholders “the application is secure” without qualification. A better statement is: “The implementation contains strong security architecture and locally tested controls, but production security still depends on edge configuration, secret management, operational controls, physical Pi assumptions, staging evidence, and independent review.”

17. Testing and evidence

Layer	What exists	What it proves	What it does not prove
Domain tests	Tenant/device/content/identity/licence/schedule state-machine tests.	Pure business rules and exact boundary behavior.	HTTP, PostgreSQL, OS, or hardware behavior.
Agent tests	Options, atomic state/key handling, inventory, cache verification, corruption rejection, resume.	Important local failure and integrity behavior.	Physical Pi, systemd, Chromium, GPU, or network edge.
Integration tests	ASP.NET hosting, PostgreSQL 18/Testcontainers, RLS, migrations, identity flow, crypto, storage.	Real database and key security boundaries on the test environment.	Production infrastructure and public TLS/proxy behavior.

Layer	What exists	What it proves	What it does not prove
React tests	Authentication/dashboard and player safe-state behavior.	Component-visible behavior in jsdom.	Real browser rendering and complete backend integration.
Playwright/axe	Five Chrome scenarios for admin requests/accessibility and fail-closed player behavior.	Real browser behavior for intercepted deterministic states and automated WCAG checks.	Full live backend or manual keyboard/screen-reader acceptance.
Field readiness	Virtual Windows device and Linux ARM64 package generation.	Real enrollment/heartbeat/synchronization logic on workstation and publishability.	Actual Raspberry Pi/hardware acceptance.

Recorded local result

- 85/85 .NET tests: 44 domain, 11 agent, 30 integration.
- 7/7 React tests.
- 5/5 Playwright/axe scenarios.
- Two Vite builds, lint, and TypeScript type checking recorded successful.
- PostgreSQL RLS and a real Kestrel HTTPS client-certificate path recorded successful.

Always attach date, commit, environment, command, output, and artifact hash when presenting these as formal evidence. A planned test is not an executed test.

18. Development and deployment

18.1 Developer prerequisites

- .NET SDK 10.0.203 exactly.
- Node.js 24 or 25 and npm 11.
- Docker Desktop/Engine with Compose v2.
- Git and PowerShell 5.1+ locally.

18.2 Local services and secrets

Docker Compose exposes PostgreSQL and ClamAV only on loopback. The API refuses to start outside testing when essential configuration is missing or unsafe. Required production-style inputs include:

- restricted PostgreSQL connection string;
- absolute Data Protection key directory;
- at least 32 random bytes of token-digest pepper;
- device CA PFX and password;
- encrypted ECDSA licence-signing key and password;
- absolute private content root;
- ClamAV endpoint and offline-allowance setting.

The development helper creates local-only security material beneath ignored `.data`. It is not a production key-management solution.

18.3 Database identities

- **Migration owner:** owns schema and applies reviewed migrations; unavailable to the web process.
- **Web runtime:** no superuser/owner/BYPASSRLS; narrow table operations; can insert but not read notification payloads.
- **Notification worker:** separate connection/role; only select/update notification rows under a special RLS context.
- Production backup and monitoring identities must also be separate.

18.4 Pi release and installation

- 1 Build the React player.
- 2 Publish the .NET agent self-contained for `linux-arm64`, embedding player assets.
- 3 Create a versioned `tar.gz` and SHA-256 file.
- 4 On aarch64 Raspberry Pi OS, run the root installer with server URL, trusted digest, version, enrollment-code file, optional field CA, and graphical kiosk user.
- 5 The installer verifies architecture and artifact SHA-256, creates service identities/directories, installs a root-owned immutable release, and atomically updates `/opt/display-control/current`.

6

It configures agent state/cache limits and enables agent/kiosk services.

7

The enrollment file is removed after successful enrollment.

18.5 systemd hardening

The agent and kiosk run under different users. Units remove capabilities, enable `NoNewPrivileges`, restrict writable paths and address families, protect system/home/kernel/control groups, use private temporary directories, apply owner-only umask, and restart failed services. Chromium keeps its normal sandbox.

18.6 CI pipeline

The GitHub workflow checks out without persistent credentials, installs pinned SDKs, restores locked dependencies, builds/tests .NET, installs npm dependencies with `npm ci --ignore-scripts`, runs production dependency audit, lint, type checking, React tests/builds, installs Chromium, runs Playwright, validates documentation traceability, validates Compose configuration, and uploads test/failure artifacts.

19. Operations and troubleshooting

19.1 Health and logs

- API liveness: `/_health/live`.
- API readiness: `/_health/ready`.
- Pi player health: `http://127.0.0.1:8787/player/v1/health`.
- Pi service status: `systemctl status display-control-agent display-control-kiosk`.
- Pi logs: `journalctl -u display-control-agent -u display-control-kiosk`.

19.2 Safe failure behavior

Failure	Expected behavior	First checks
Network/API unavailable	Continue only already-ready content under a still-valid lease during the same boot; otherwise Not licensed.	DNS/TLS, server health, Pi time, agent journal.
Licence denied	Clear player manifest/assets and show Not licensed.	Tenant/device/certificate/licence state and UTC bounds.
No assignment	Show No content assigned under a valid lease.	Published playlist, assignment target/window/priority, group membership.
Scanner unavailable	Keep content quarantined; no approval/publication.	ClamAV health, signatures, volume, network endpoint.
Partial/corrupt download	Reject activation; retain safe state; resume valid partial on retry.	Range response, content length, SHA-256, free disk, device API logs.
Disk nearly full	Remove inactive objects if safe; fail synchronization if reserve cannot be kept.	Cache ceiling/reserve, active hashes, filesystem capacity.

Failure	Expected behavior	First checks
Chromium failure	systemd restarts kiosk; local agent remains authority.	Kiosk journal, graphical session, Wayland/X11 variables.
Agent failure	Player local API disappears; React fails to Not licensed; systemd restarts agent.	Agent journal, state/key permissions, certificate, configuration.

19.3 Operational alerts to implement

- API/database/scanner/object-storage readiness failure.
- Heartbeat missing/degraded device fleet.
- Certificate and licence expiry approaching.
- Repeated enrollment/authentication/replay rejection.
- Synchronization errors, digest failures, and cache/disk pressure.
- Notification backlog, retry age, and permanent failures.
- Backup age and restoration-test status.
- Signing/CA/Data Protection key lifecycle.

20. Current status and production gap

Area	Status	Explanation
Architecture and requirements	Implemented/documentated	Charter, requirements, acceptance criteria, traceability, ADRs, architecture, data, API, threat, tests, and operations exist.
Core backend	Implemented locally	Tenant, identity, device, licensing, content, scheduling, audit, and security boundaries are coded.
Admin UI	Core workflow implemented	Platform and tenant dashboards use real API shapes; advanced lifecycle UX is not complete.
Agent/player	Software implemented	Enrollment, mTLS, heartbeat, signed lease, cache, and local playback exist.
Automated local verification	Recorded passing	85 .NET, 7 React, and 5 browser scenarios in latest status record.
Physical hardware	Open	Pi 4/5, monitors, codecs, reboot, outage, clock, disk, and soak acceptance required.
Email	Code complete, external proof open	Real SMTP provider delivery/operations not validated.
Staging/production	Not ready	Hosting, public TLS, secrets, object storage, monitoring, backups, DR, load, security review, and deployment controls remain.
Final report/delivery	Prepared, not finalized	Final French DOCX/PDF, screenshots, identity metadata, and refreshed evidence remain.

Recommended order of work

1. Build a clean production-like staging environment with public TLS, restricted database roles, private storage, ClamAV, secrets, and SMTP.
2. Deploy a physical Pi 4/5 and execute playback/failure/offline/clock/disk/reboot/soak scenarios.
3. Add live-backend Playwright journeys, manual accessibility, load/endurance, malicious-file testing, and ASVS/penetration review.
4. Implement observability, backup/restore proof, notification operations, signed updates, and controlled rollback.
5. Freeze capacity/media/hosting decisions, refresh traceability evidence, and produce the final report/delivery package.

21. Manager talking points and questions

60-second presentation

Display Control is a secure multi-tenant digital-signage platform. Administrators manage customers, users, Raspberry Pi screens, licences, media, playlists, and schedules in a React web application. Each Pi makes an outbound connection to an ASP.NET Core API, authenticates with its own certificate, verifies a signed time-limited licence decision, downloads only approved content, checks every file, and plays it locally through Chromium. The main secure workflow works locally; the remaining work is physical hardware validation and production operations/security evidence.

Five points to emphasize

1. **It is more than a media player:** it manages authorization, identity, tenant isolation, content safety, synchronization, and operational state.
2. **The Pi is outbound-only:** customer routers do not need inbound access.
3. **Offline is bounded:** cached content cannot be authorized indefinitely; the lease is capped at 24 hours and other earlier boundaries.
4. **Security is layered:** roles, MFA, sessions, CSRF, mTLS, RLS, private storage, malware scanning, signatures, hashes, and restricted services work together.
5. **Status is honest:** the vertical slice is implemented and locally tested, but physical and production evidence remain required.

Likely questions and suggested answers

Question	Suggested answer
Why not microservices?	A modular monolith reduces deployment and transactional complexity while preserving clear internal boundaries. It is appropriate for the current scope and team size.
How are customers isolated?	The session establishes tenant context; authorization checks it; EF filters it; PostgreSQL forced RLS enforces it; same-tenant foreign keys prevent cross-tenant relationships.
What happens when Internet fails?	Already-ready cached playback can continue only while a signed lease remains valid during the same boot. When authorization cannot be proven, the device shows Not licensed.
Can a user copy content URLs?	Media is private and not in the public web root. The Pi receives only device-scoped resources, and the browser sees only loopback URLs currently authorized by the agent.
What if a Pi is stolen?	Certificates/devices can be revoked, but a determined person with root/physical access may extract software-held credentials/cache. Hardware-backed security is outside the current baseline.
Is it production-ready?	No. The architecture and core workflow are strong, but production infrastructure, physical Pi acceptance, operational controls, load/security testing, and final evidence remain.
What is the biggest technical strength?	The authorization chain: isolated tenant data leads to a signed device/certificate/manifest-bound lease that the Pi independently verifies before exposing local content.
What is the biggest project risk?	Moving from locally verified software to a production-operated system: real hardware, public TLS/edge, secrets, storage, monitoring, backup/restore, capacity, and independent security evidence.

Things not to claim yet

- "Production-ready," "penetration-tested," or "fully secure."
- Validated Pi 4/5 GPU/video performance.
- Validated real-email delivery.
- Proven production-scale fleet capacity.
- Proven disaster recovery or zero data loss.
- Complete WCAG conformance based only on automated axe checks.

22. Glossary

Term	Plain-language meaning
API	The server interface used by the admin browser and Pi agent.
Tenant	One isolated customer organization.
Multi-tenant	One platform serving several customers while isolating their data and permissions.
Modular monolith	One deployed backend divided internally into clear functional modules.
SPA	Single-page web application; React updates views without full page reloads.
RLS	PostgreSQL Row-Level Security; database policies that filter/reject rows by tenant context.
MFA/TOTP	Multifactor authentication using rotating six-digit authenticator-app codes.
CSRF	An attack that tricks a signed-in browser into sending an unwanted request; antiforgery tokens defend against it.
mTLS	Mutual TLS; both server and device present cryptographic certificates.
CSR	Certificate-signing request containing a public key and proof; the private key stays on the Pi.
ECDSA P-256 / ES256	Elliptic-curve signing algorithm used for device certificates and licence leases.
SHA-256	A cryptographic digest used to detect changed/corrupt content and identify keys/tokens safely.
Lease	A signed, time-bounded proof that a specific device/certificate may use a specific desired state.
Desired state	The exact immutable playlist manifest selected for one device.
Fail closed	Deny access or playback when the system cannot safely prove authorization.
Immutable	Published bytes/versions are not modified; a new version must be created.
Idempotency	Safely retrying an operation without accidentally applying it twice.
Concurrency token	A value that detects whether a resource changed since a user loaded it.
ClamAV	Antivirus engine used to scan uploaded content.
Data Protection	ASP.NET Core facility used to encrypt/authenticate stored MFA and notification payloads.
systemd	Linux service manager that starts, restricts, monitors, and restarts the Pi agent/kiosk.
Wayland/X11	Linux graphical display systems that Chromium may use on Raspberry Pi OS.
CI	Continuous integration: automated clean build, tests, checks, and retained evidence.
DR	Disaster recovery: restoring service and data after a serious failure.
ASVS	OWASP Application Security Verification Standard, used for structured security review.

23. Authoritative repository references

- `README.md` — project overview and repository map.
- `docs/00-project/ETAT_AVANCEMENT.md` — current French status and honest remaining work.
- `docs/02-architecture/ARCHITECTURE.md` — system/container/request/device architecture.
- `docs/02-architecture/DATA_MODEL.md` — entities, relationships, and invariants.
- `docs/02-architecture/API_CONTRACT.md` — route and protocol outline.
- `docs/02-architecture/THREAT_MODEL.md` — assets, threats, trust boundaries, and controls.
- `docs/03-testing/TEST_STRATEGY.md` — layered testing and release gates.
- `docs/05-operations/DEVELOPMENT.md` — local setup and required secrets.
- `docs/05-operations/DATABASE_SECURITY.md` — PostgreSQL roles and tenant context.
- `docs/05-operations/OBJECT_STORAGE.md` — private storage boundary and production gate.
- `docs/05-operations/PI_PROVISIONING.md` — Pi packaging/install/security baseline.
- `src/DisplayControl.Api/Program.cs` — runtime configuration and security pipeline.
- `src/DisplayControl.DeviceAgent` — enrollment, heartbeat, lease, cache, and player authority.

Final takeaway: The core product is a controlled chain of trust, not simply a screen player. User authority, tenant-isolated data, approved immutable content, device identity, signed bounded permission, verified cache, and local playback all have to agree before content appears.